

The `handleEvent()` method then does a few more checks to ensure that the number being entered is legal (for example, doesn't have two periods, only has one minus sign and, so on). Whenever a valid key press is detected, the code calls the `handleEvent()` method of the `InputText` base class to do whatever it needs to do with that key (display or edit the field).

When the user presses RETURN or ENTER, client code calls the `getValue()` method ④ to get the user's entry. The `getValue()` method in this class calls `getValue()` in the `InputText` class to get the string from the field, then attempts to convert that string to a number. If that conversion fails, it raises an exception.

By overriding methods, we have built a very powerful new reusable class that extends the functionality of the `InputText` class, without changing a single line in the base class. `InputText` will continue to function as a class by itself, without any changes to its functionality whatsoever.

DisplayMoney

As a second real-world example, we'll create a field to display an amount of money. To make this general, we'll display the amount with a chosen currency symbol, place that currency symbol to the left or the right of the text (as appropriate), and format the number by adding commas between every three digits, followed by a period and then two decimal digits. For example, we would like to be able to display 1234.56 US dollars as \$1,234.56.

The `pygwidgets` package already has a `DisplayText` class. We can instantiate an object from that class using the following interface:

```
def __init__(self, window, loc=(0, 0), value='',
             fontName=None, fontSize=18, width=None, height=None,
             textColor=PYGWIDGETS_BLACK, backgroundColor=None,
             justified='left', nickname=None):
```

Let's assume that we have some code that creates a `DisplayText` object named `oSomeDisplayText` using the appropriate arguments. Any time that we want to update the text in a `DisplayText` object, we must call its `setValue()` method, like this: